

1.

- IP (Internet Protocol)

定义与作用:

IP是网络层协议，主要负责在互联网中把数据包从源地址传输到目的地址。当你在浏览器中访问一个网站时，你的设备会把请求数据打包，并通过IP协议添加源IP（你的设备的IP地址）和目的IP（网站服务器的IP地址），然后发送出去。

IP地址格式

常见的有IPv4和IPv6两种格式。IPv4地址是32位的二进制数，通常用点分十进制表示，IPv6地址是128位的二进制数，格式更加复杂

- TCP (Transmission Control Protocol)

定义与作用:

TCP是一种传输层协议，它建立在IP协议之上。TCP的主要功能是提供可靠的、面向连接的通信服务。

在文件传输场景中，TCP会确保文件的每个部分都准确无误地从发送端传输到接收端

- TCP和IP的关系:

TCP/IP协议族是一组协同工作的协议集合。IP负责将数据包在网络中路由到正确的位置，而TCP则负责在两个端点之间建立可靠的连接，保证数据的准确传输。

2. I/O (Input/Output) 流是Java中用于处理输入和输出操作的一种机制。

- 输入流 (InputStream/Reader)：用于从数据源读取数据到程序中。比如从文件中读取内容到内存，或者从网络连接接收数据。
- 输出流 (OutputStream/Writer)：用于把程序中的数据输出到目标位置。例如把内存中的数据写入文件，或者通过网络连接发送数据。
- 字节流 (InputStream/OutputStream)：以字节为单位处理数据，适合处理二进制数据
- 字符流 (Reader/Writer)：以字符为单位处理数据，更适合处理文本数据，因为它会考虑字符编码。
- Reader：抽象类，常见的实现类如 FileReader（用于读取文本文件）、BufferedReader（可以高效地读取文本行）等。
- Writer：抽象类，常见的实现类如 FileWriter（用于写入文本文件）、PrintWriter（提供格式化的写入功能）等。

3. URL (Uniform Resource Locator)，即统一资源定位符，用于定位互联网上的资源。它是一个字符串，包含了访问某个资源所需的协议、主机名、端口号、路径等信息。

常见的URL格式是 <http://www.example.com:8080/index.html>。其中 http 是协议，www.example.com 是主机名，8080 是端口号，index.html 是资源路径。

运用URL:

- 创建URL对象
可以通过将URL字符串传递给 URL 构造函数来创建一个 URL 对象。
- 获取URL的各个部分信息
可以使用 URL 对象的方法获取URL的各个组成部分。
- 读取URL指向的资源内容（以文本为例）
可以通过打开连接并获取输入流来读取内容。

4. Socket:

```
import java.io.BufferedReader;  
import java.io.IOException;  
import java.io.InputStreamReader;
```

```

import java.io.PrintWriter;
import java.net.Socket;

public class SocketExample {
    public static void main(String[] args) {
        try {

            // 创建与服务器的连接
            Socket socket = new Socket("127.0.0.1", 5000);
            // 发送信息
            PrintWriter writer = new PrintWriter(socket.getOutputStream(), true);
            writer.println("message to send");
            writer.print("another message");

            // 接收信息
            BufferedReader reader = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
            String message = reader.readLine();
            System.out.println("Received message: " + message);

            // 关闭连接
            socket.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}

```

5. 一、创建客户端连接

创建 Socket 对象

通过指定服务器的 IP 地址和端口号来创建一个 Socket 对象，这将发起一个到服务器的连接请求。一旦连接建立，可以通过 Socket 的输出流来发送数据。

二、创建服务器端监听

-创建 ServerSocket 对象

指定一个端口号来创建 ServerSocket 对象，服务器将在这个端口上监听客户端的连接请求。

-接受客户端连接

使用 ServerSocket 的 accept() 方法来等待并接受客户端的连接请求。这个方法将阻塞，直到有一个客户端连接到来。

-接收数据

一旦客户端连接成功，就可以通过客户端 Socket 的输入流来接收数据。

三、关闭连接

在通信完成后，应该关闭 Socket 连接以释放资源。